AFS
Agile Full Stack
Developer Bootcamp
Thoughtworks @

Code Review

/thoughtworks

© 2024 Thoughtworks | Restricted

Question: Why do we need Code review?



© 2021 Thoughtworks

Code Review Value

Code Assurance

Improving Code Quality

Reviewers assess code for readability, maintainability, and adherence to coding standards, ensuring a consistent and robust codebase.

Reduce potential defects

Reviewers identify logic errors, edge cases, and possible performance or security issues early in the development process, preventing defects from reaching production.

© 2024 Thoughtworks, Inc. Commercial in Confidence

Team Cognition

Knowledge Sharing

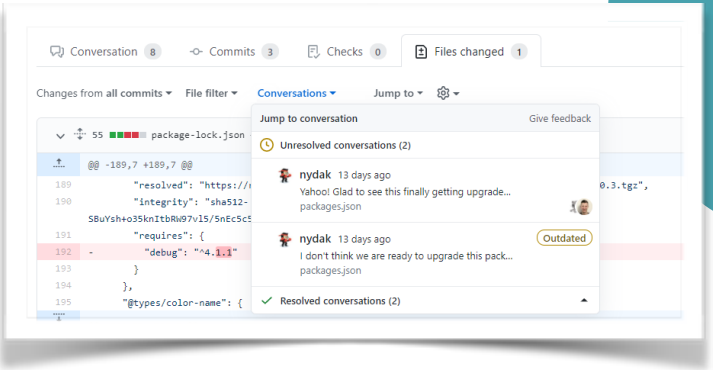
Code reviews expose developers to different coding styles, approaches, and potential solutions, fostering a shared understanding of the project.

Skills Learning and Growth

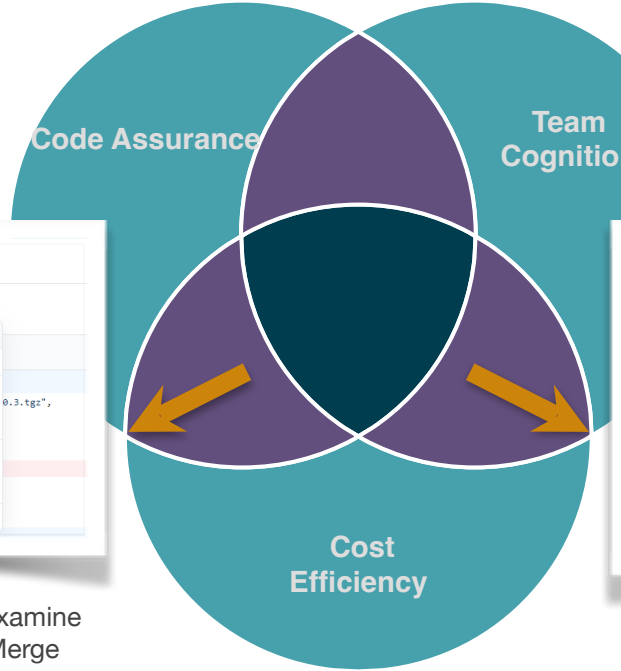
Developers gain insights into new techniques, tools, and best practices through the review process.

Code Review “Impossible Triangle”

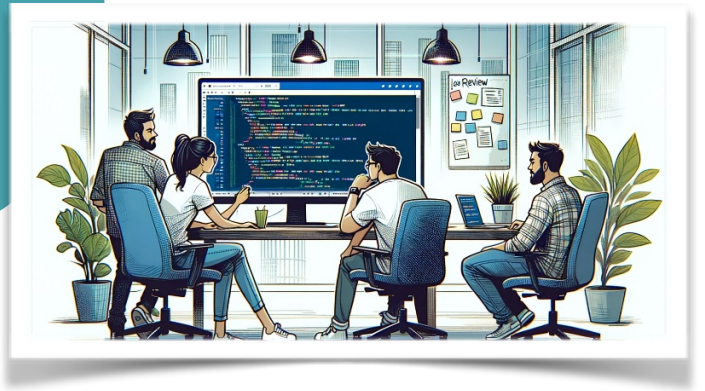
Change-based Code Review



An **asynchronous** review process where peers examine a set of code changes (e.g., via Pull Request or Merge Request) to ensure quality, consistency, and defect prevention before integration.



Collective Code Review



Conducting it at a fixed time with **full participation** is more conducive to reaching consensus and sharing knowledge.

© 2021 Thoughtworks

AFS
Agile Full Stack
Developer Bootcamp
Thoughtworks @

Change-based Code Review

/thoughtworks

© 2025 Thoughtworks | Restricted

Merge Request Process

Process:

1. Code Push
2. Code Scan
3. TL Merge Request Review
4. Code Merge

Code Style Guide



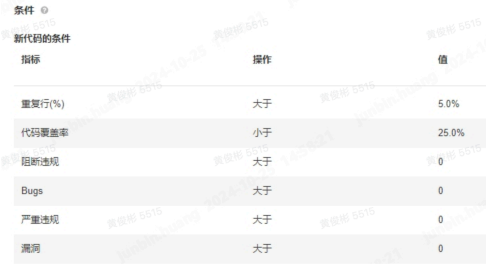
Check List

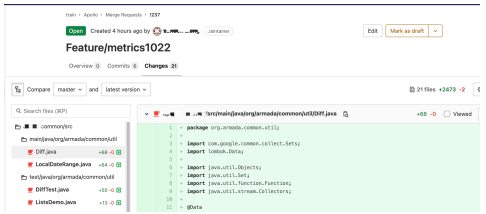


Code Review

- Functionality correctness
- Performance considerations
- Test coverage
- Error and exception handling

Code Push | **Code Scan** | **TL Review** | Code Merge





© 2024 Thoughtworks | Restricted

Change-based Code Review has several drawbacks

- **Overly heavy in large teams**

  When the team size is 3 people or less, the process tends to be overly heavy.

- **Bottlenecks in large teams**

  When the team size exceeds 9 people, reviewers will become a bottleneck.

- **Knowledge silos**

  Regular collective code reviews are still necessary; otherwise, effective team knowledge sharing cannot be achieved.

- **Technical debt management**

  The process only has two statuses: pass and fail. However, issues that cannot be fixed in a timely manner still need to be included in technical debt management.

- **Delayed feedback**

  Issues are often identified only after the code is written and submitted, making fixes more costly and time-consuming than catching problems during development.

- **Limited collaboration**

  Unlike real-time pairing, there's less interactive discussion, reducing opportunities for collective problem-solving or idea refinement.

© 2025 Thoughtworks | Restricted

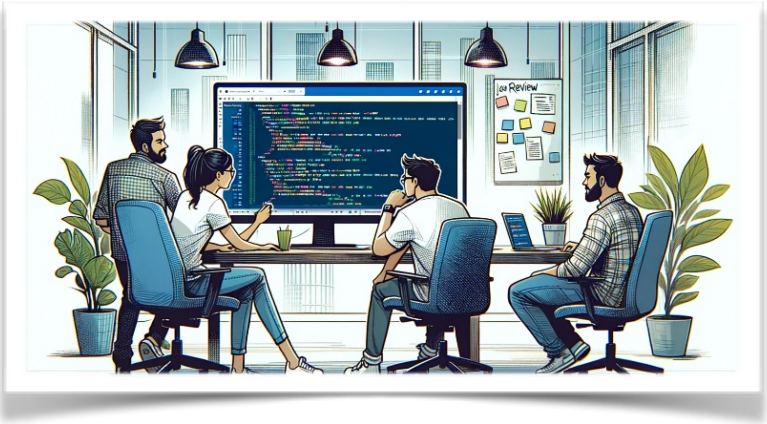
AFS
Agile Full Stack
Developer Bootcamp
Thoughtworks @

Collective Code Review

/thoughtworks

© 2025 Thoughtworks | Restricted

Collective Code Review Process



Process:

1. **Submitter:** Present the change context
2. **Submitter:** Open and walk through the commits
3. **Reviewers:** Provide feedback

Organizer:

- **Facilitate and keep time**
- Record feedback issues;

Specific implementation:

Participants：Development team

Frequency：Daily or 2-3 times a week

Time Box：30 minutes to 1 hour

Notes:

- The organizer should be rotated regularly. Organizing Code Review activities is not one person's responsibility, but the **collective responsibility** of the team.
- Improvements that cannot be made in a timely manner should be included in **technical debt** management.

© 2025 Thoughtworks | Restricted

Collective Code Review has several drawbacks

- **Time-boxed meetings may limit review depth.**

  Time-boxed meetings often leave little room for line-by-line inspection, so subtle issues may be missed.

- **Can be inefficient if not well-facilitated.**

  Involving multiple people at once can be costly in terms of time, especially if discussion drifts off-topic.

- **Herd Effect**

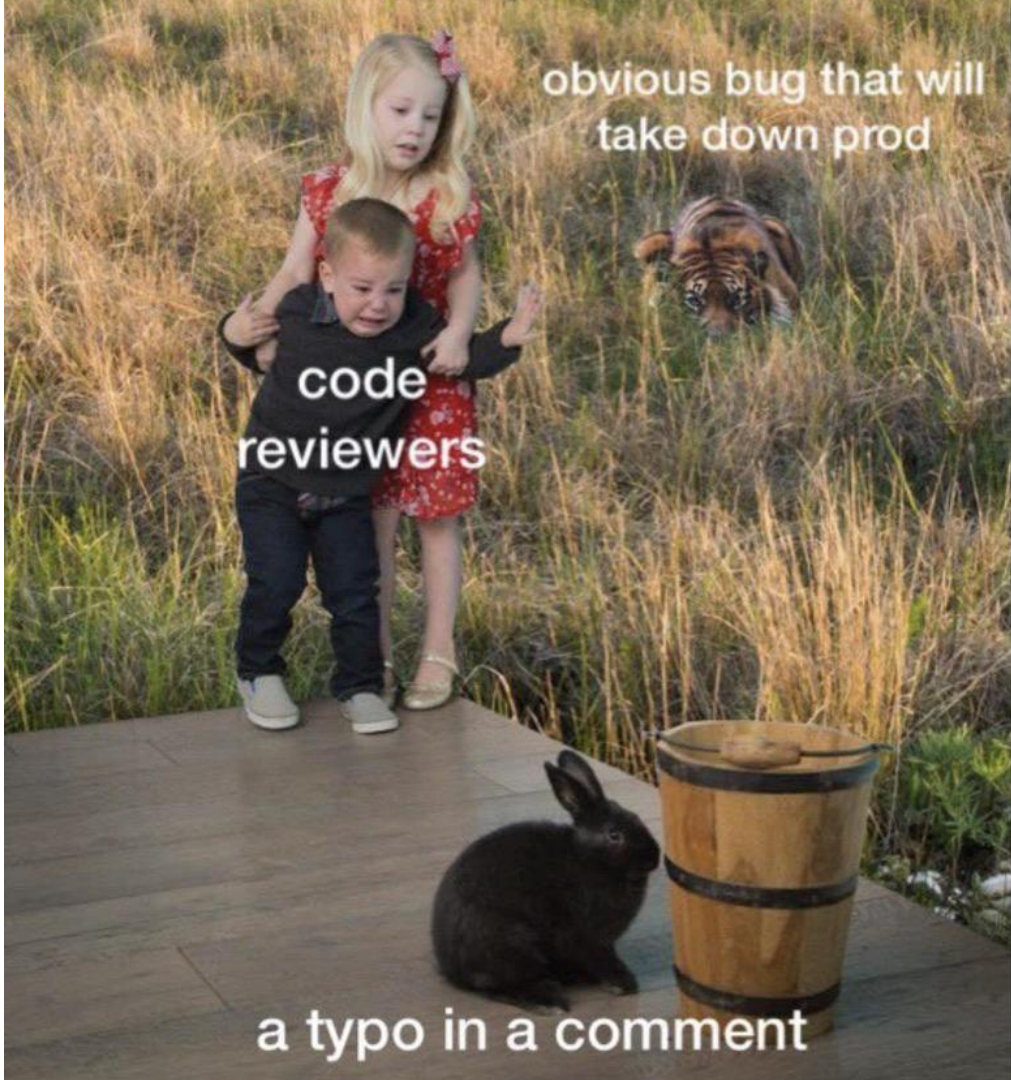
  Group settings may discourage dissenting opinions, leading to superficial agreement instead of critical review.

© 2025 Thoughtworks | Restricted

Best Practices

© 2025 Thoughtworks | Restricted

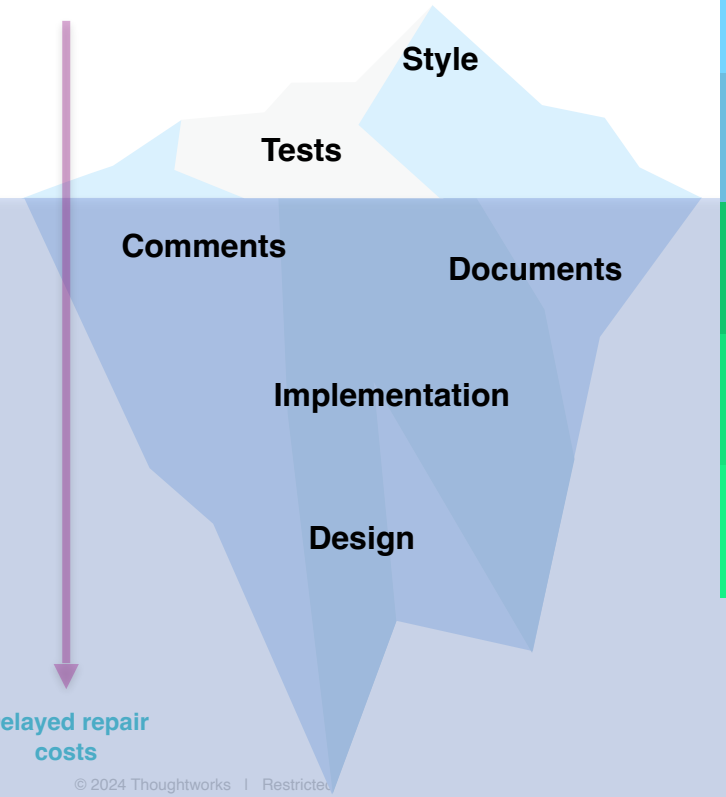
What Do Code Reviewers Look For?



© 2024 Thoughtworks | Restricted

Attention Bias in Code Review

Reviewers tend to focus on *obvious, surface-level issues* (style, comments, typos) while neglecting *deep, high-cost issues* (logic, design, complexity) — exactly like the iceberg model.



Coding Style, naming, duplication, readability

Functional, been tested

User guides, tutorials, documents

Correct logic, appropriate complexity, robust

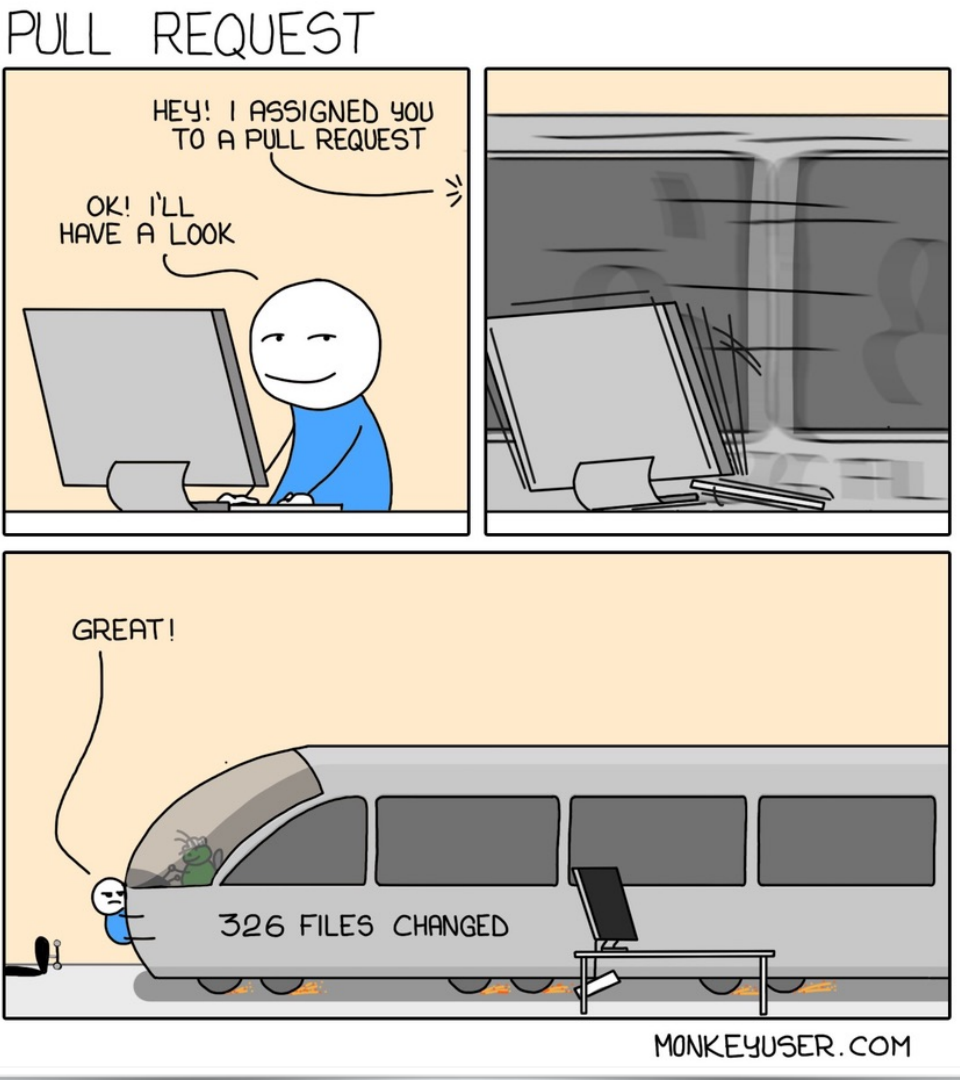
Design principles, design patterns, and API design

- Establish coding **standards**
- Take advantage of **automation**

Preferred “deeper” debt

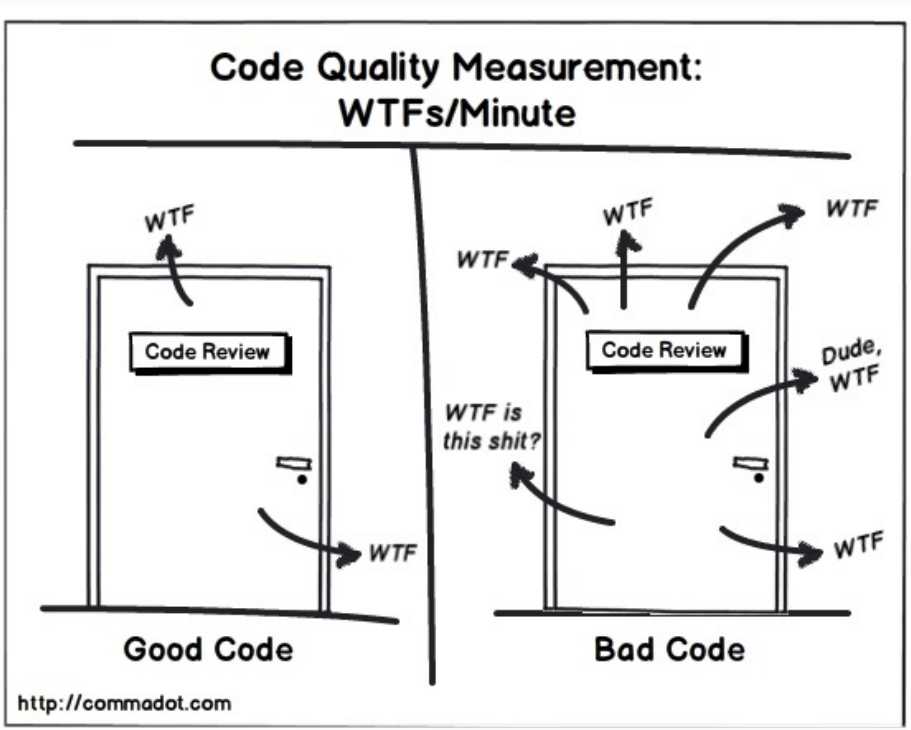
Pay more attention to code issues that are more expensive to fix later instead of more obvious.

© 2024 Thoughtworks | Restricted



Step small

Small (≤400 LoC), frequent code reviews are more efficient



Provide constructive feedback

Focus on the code, not the person, and provide **specific**, **actionable** comments

© 2024 Thoughtworks | Restricted

AFS
Agile Full Stack
Developer Bootcamp
Thoughtworks @

Homework Review



30 mins

/thoughtworks

© 2025 Thoughtworks | Restricted